



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

Experiment No. 8
Implement Ant Colony Optimization
Date of Performance:
Date of Submission:



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

2024-2025

Aim: To implement the Ant Colony Optimization (ACO) algorithm.

Theory:

Ant Colony Optimization (ACO) is inspired by the behavior of ants finding the shortest path using pheromones.

- Steps:

1. Initialize pheromone levels.
2. Simulate ant movement where they probabilistically choose paths based on pheromone levels and heuristic desirability.
3. Update pheromone levels based on the quality of solutions found.
4. Repeat until convergence.

- Applications: Traveling salesman problem, routing, and scheduling.

Procedure/Algorithm:

1. Define the optimization problem as a graph.
2. Initialize pheromone levels on edges.
3. Set parameters for pheromone evaporation and deposition.
4. Repeat for a specified number of iterations:
 - o Simulate ant movement based on pheromone levels and heuristic information.
 - o Update pheromone levels based on ant paths.
5. Display the best solution found.

Code:

```
import numpy as np
import random

class AntColonyTSP:
    def __init__(self, distance_matrix, num_ants, num_iterations, alpha, beta, evaporation_rate, q0):
        self.distance_matrix = np.array(distance_matrix)
        self.num_ants = num_ants
        self.num_iterations = num_iterations
        self.num_cities = len(distance_matrix)
        self.alpha = alpha
        self.beta = beta
        self.evaporation_rate = evaporation_rate
        self.q0 = q0
        self.pheromone = np.ones((self.num_cities, self.num_cities)) * 0.1
```



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

```
def run(self):
    best_path = None
    best_cost = float('inf')

    for iteration in range(self.num_iterations):
        all_paths = []
        all_costs = []

        for _ in range(self.num_ants):
            path, cost = self.construct_solution()
            all_paths.append(path)
            all_costs.append(cost)

        best_iteration_cost = min(all_costs)
        best_iteration_path = all_paths[all_costs.index(best_iteration_cost)]

        if best_iteration_cost < best_cost:
            best_cost = best_iteration_cost
            best_path = best_iteration_path

        self.update_pheromone(all_paths, all_costs)

        print(f"Iteration {iteration + 1}: Best path {best_iteration_path} with cost: {best_iteration_cost}")

    print(f"Best path found: {best_path} with cost: {best_cost}")

def construct_solution(self):
    path = [0] # Start at city 0
    visited = set(path)
    total_cost = 0

    for _ in range(self.num_cities - 1):
        current_city = path[-1]
        next_city = self.select_next_city(current_city, visited)
        path.append(next_city)
        visited.add(next_city)
        total_cost += self.distance_matrix[current_city][next_city]

    total_cost += self.distance_matrix[path[-1]][path[0]] # Return to start
    return path, total_cost

def select_next_city(self, current_city, visited):
    probabilities = self.calculate_probabilities(current_city, visited)
```



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

```
q = random.random()

if q < self.q0:
    return np.argmax(probabilities) # Exploitation
else:
    return np.random.choice(range(self.num_cities), p=probabilities) # Exploration

def calculate_probabilities(self, current_city, visited):
    tau = self.pheromone[current_city]
    eta = 1 / (self.distance_matrix[current_city] + np.finfo(float).eps)
    for v in visited:
        eta[v] = 0
    pheromone_power = tau ** self.alpha
    distance_power = eta ** self.beta
    numerator = pheromone_power * distance_power
    denominator = np.sum(numerator)
    return numerator / denominator if denominator > 0 else np.ones(self.num_cities) / self.num_cities

def update_pheromone(self, all_paths, all_costs):
    self.pheromone *= (1 - self.evaporation_rate) # Evaporation
    for path, cost in zip(all_paths, all_costs):
        pheromone_deposit = 1 / cost # More pheromone for shorter paths
        for i in range(self.num_cities - 1):
            self.pheromone[path[i]][path[i + 1]] += pheromone_deposit
        self.pheromone[path[-1]][path[0]] += pheromone_deposit # Return trip

distance_matrix = [
    [0, 4, 8, 6, 5], # A
    [4, 0, 3, 7, 2], # B
    [8, 3, 0, 4, 6], # C
    [6, 7, 4, 0, 3], # D
    [5, 2, 6, 3, 0] # E
]

num_ants = 5
num_iterations = 15
alpha = 1.2 # Influence of pheromone
beta = 2.5 # Influence of distance
evaporation_rate = 0.4 # Higher evaporation for dynamic updates
q0 = 0.7 # More exploitation

aco = AntColonyTSP(distance_matrix, num_ants, num_iterations, alpha, beta, evaporation_rate, q0)
aco.run()
```



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

Output:

Iteration 1: Best path [0, 4, 1, 2, 3] with cost: 20
Iteration 2: Best path [0, 4, 3, 2, 1] with cost: 19
Iteration 3: Best path [0, 1, 4, 3, 2] with cost: 21
Iteration 4: Best path [0, 4, 3, 2, 1] with cost: 19
Iteration 5: Best path [0, 1, 4, 3, 2] with cost: 21
Iteration 6: Best path [0, 1, 4, 3, 2] with cost: 21
Iteration 7: Best path [0, 1, 4, 3, 2] with cost: 21
Iteration 8: Best path [0, 1, 4, 3, 2] with cost: 21
Iteration 9: Best path [0, 1, 4, 3, 2] with cost: 21
Iteration 10: Best path [0, 1, 4, 3, 2] with cost: 21
Iteration 11: Best path [0, 1, 4, 3, 2] with cost: 21
Iteration 12: Best path [0, 1, 4, 3, 2] with cost: 21
Iteration 13: Best path [0, 1, 4, 3, 2] with cost: 21
Iteration 14: Best path [0, 1, 4, 3, 2] with cost: 21
Iteration 15: Best path [0, 1, 4, 3, 2] with cost: 21
Best path found: [0, 4, 3, 2, 1] with cost: 19

Conclusion: In conclusion, the Ant Colony Optimization (ACO) algorithm efficiently solves optimization problems like the Traveling Salesman Problem by simulating natural ant behavior to find the shortest path through pheromone-based exploration and exploitation.

Answer the Following Question:

1. What is the role of pheromones in ACO?

In Ant Colony Optimization (ACO), pheromones play a crucial role in guiding the ants toward the optimal solution. Ants leave a trail of pheromones on the paths they take, and the intensity of the pheromone trail influences the probability that other ants will follow the same path. Over time, paths with stronger pheromone concentrations are more likely to be chosen, reinforcing good solutions. Additionally, pheromones evaporate over time, allowing the algorithm to forget suboptimal paths and explore new possibilities. This dynamic balance of exploration and exploitation helps the algorithm converge toward the best solution.

2. How does ACO handle the exploration-exploitation trade-off?

ACO balances exploration and exploitation using pheromones and randomness. When ants find paths, they leave pheromones, attracting more ants to good solutions (exploitation). However, some ants still explore new paths randomly to avoid getting stuck in one solution (exploration). Over time, pheromone evaporation removes weaker paths, ensuring the algorithm keeps improving. This way, ACO finds the best solution by learning from past paths while still searching for better ones.